LÝ THUYẾT ĐỒ THỊ

Tất cả về đường đi trong đồ thị

Nhóm 6

Nguyễn Văn Phú **Hà Minh Đức** **Nguyễn Tấn Đạt**

THPT Chuyên ĐHV

Ngày 18 tháng 3 năm 2026

Mục lục

1. Khái quát
2. PATH by DFS
3. PATH by BFS
4. Thuật Floyd-Warshall
5. Thuật Dijkstra
6. Tổng kết

Khái quát

Khái niệm:

Đường đi (path) là một dãy các đỉnh liên tiếp trong đồ thị sao cho mỗi cặp đỉnh kề đều có cạnh nối giữa chúng.

Khái quát

Khái niệm:

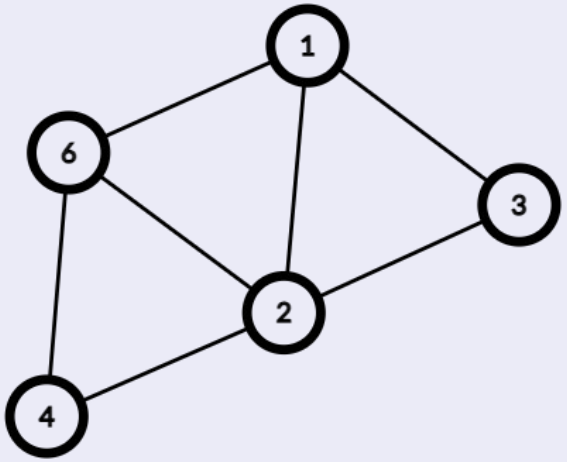
Đường đi (path) là một dãy các đỉnh liên tiếp trong đồ thị sao cho mỗi cặp đỉnh kề đều có cạnh nối giữa chúng.

Các dạng đường đi thường gặp

- → **Đường đi đơn (simple path)**: Là đường đi không lặp lại đỉnh nào
- → **Chu trình (cycle)**: Đường đi bắt đầu và kết thúc ở cùng một đỉnh
- → **Đường đi trọng số**: Mỗi cạnh có trọng số, tổng trọng số = tổng chi phí đường đi.

Ví dụ về 1 đường đi trong đồ thị

Ví dụ:



- Đường đi $3 \to 1 \to 6 \to 4$ là một đường đi đơn
- Đường đi $3 \to 1 \to 2 \to 3$ là một ví dụ cho chu trình
- Còn ví dụ về đường đi có trọng số chúng ta sẽ nói rõ hơn vào phần sau

PATH by DFS

- Như chúng ta đã biết, thuật toán DFS trong đồ thị cho phép duyệt và đi đến toàn bộ đỉnh của đồ thị 1 lần.

PATH by DFS

- Như chúng ta đã biết, thuật toán DFS trong đồ thị cho phép duyệt và đi đến toàn bộ đỉnh của đồ thị 1 lần.
- Nhưng chỉ hàm DFS thông thường thì chúng ta chỉ có thể làm các bài đơn giản như check 1 đỉnh có thể đến được từ 1 đỉnh khác hay không. Nhưng trong chuyên đề này chúng ta cần nhiều hơn thế từ nó.

PATH by DFS

- Như chúng ta đã biết, thuật toán DFS trong đồ thị cho phép duyệt và đi đến toàn bộ đỉnh của đồ thị 1 lần.
- Nhưng chỉ hàm DFS thông thường thì chúng ta chỉ có thể làm các bài đơn giản như check 1 đỉnh có thể đến được từ 1 đỉnh khác hay không. Nhưng trong chuyên đề này chúng ta cần nhiều hơn thế từ nó.
- Để có thể tìm được đường đi từ đỉnh u đến đỉnh v (giả sử) bằng DFS, chúng ta cần thêm 1 mảng par[], với par[i] là đỉnh cha của đỉnh i khi duyệt dfs từ đỉnh nguồn u

PATH by DFS

- Như chúng ta đã biết, thuật toán DFS trong đồ thị cho phép duyệt và đi đến toàn bộ đỉnh của đồ thị 1 lần.
- Nhưng chỉ hàm DFS thông thường thì chúng ta chỉ có thể làm các bài đơn giản như check 1 đỉnh có thể đến được từ 1 đỉnh khác hay không. Nhưng trong chuyên đề này chúng ta cần nhiều hơn thế từ nó.
- Để có thể tìm được đường đi từ đỉnh u đến đỉnh v (giả sử) bằng DFS, chúng ta cần thêm 1 mảng par[], với par[i] là đỉnh cha của đỉnh i khi duyệt dfs từ đỉnh nguồn u
- Bên cạnh đó, nếu chỉ mỗi mảng par thì vẫn chưa đủ, chúng ta cần phải có 1 hàm trace để có thể lần lại đường đi đã được thực hiện bởi DFS

PATH by DFS

Mã giả

```
bool visite [N];
int parent[N];
vector<int> adj[N];
void dfs(int u) {
    visite [u] = true;
    for(int v : adj[u]) {
        if(!visite [v]) {
            parent[v] = u;
            dfs(v);
        }
    }
}

vector<int> path(int v) {
    vector<int> res;
    while(v != s) {
        res.eb(v);
        v = parent[v];
    }
    revers (res.begin(), res.end());
    return res;
}
```

PATH by DFS

→ Như chúng ta có thể nhận thấy từ mã giả, việc thăm được kiểm soát bởi mảng visited, vì vậy DFS sẽ lấy duy nhất 1 đường đi từ u đến v được duyệt trước mặc dù trong 1 đồ thị thì có thể có rất nhiều đường đi từ u đến v.

PATH by DFS

- → Như chúng ta có thể nhận thấy từ mã giả, việc thăm được kiểm soát bởi mảng visited, vì vậy DFS sẽ lấy duy nhất 1 đường đi từ u đến v được duyệt trước mặc dù trong 1 đồ thị thì có thể có rất nhiều đường đi từ u đến v.
- → Đường đi được trace lại phải lấy đảo ngược vì chúng ta đang đi ngược từ đỉnh đến đến đỉnh ban đầu

PATH by DFS

- → Như chúng ta có thể nhận thấy từ mã giả, việc thăm được kiểm soát bởi mảng visited, vì vậy DFS sẽ lấy duy nhất 1 đường đi từ u đến v được duyệt trước mặc dù trong 1 đồ thị thì có thể có rất nhiều đường đi từ u đến v.
- → Đường đi được trace lại phải lấy đảo ngược vì chúng ta đang đi ngược từ đỉnh đến đến đỉnh ban đầu
- → Nếu muốn thì bạn có thể sort trước mảng danh sách kề, điều đó sẽ làm cho đường đi bạn lấy được là đường đi có thứ tự từ điển nhỏ nhất, còn nếu không, nó chỉ cho ra 1 đường đi bất kì phụ thuộc vào mảng danh sách kề của bạn hiện tại

PATH by BFS

Khái quát

→ Nhìn chung, DFS và BFS không khác nhau trong việc nó chỉ duyệt qua tất cả các đỉnh có thể thăm từ 1 đỉnh bất kì, chúng khác nhau theo cách duyệt khi 1 cái là duyệt theo chiều sâu còn 1 cái là duyệt theo chiều rộng.

PATH by BFS

Khái quát

→ Nhìn chung, DFS và BFS không khác nhau trong việc nó chỉ duyệt qua tất cả các đỉnh có thể thăm từ 1 đỉnh bất kì, chúng khác nhau theo cách duyệt khi 1 cái là duyệt theo chiều sâu còn 1 cái là duyệt theo chiều rộng.

→ Vì vậy, ở đây chúng ta vẫn cần ở BFS 1 mảng par và 1 hàm trace để có thể tìm được đường đi trên đồ thị bằng nó

PATH by BFS

Khái quát

→ Nhìn chung, DFS và BFS không khác nhau trong việc nó chỉ duyệt qua tất cả các đỉnh có thể thăm từ 1 đỉnh bất kì, chúng khác nhau theo cách duyệt khi 1 cái là duyệt theo chiều sâu còn 1 cái là duyệt theo chiều rộng.

→ Vì vậy, ở đây chúng ta vẫn cần ở BFS 1 mảng par và 1 hàm trace để có thể tìm được đường đi trên đồ thị bằng nó

→ tương tự với DFS, code BFS trong vấn đề này không quá phức tạp khi cách trace và vis cũng tương tự như DFS, mã giả như sau:

PATH by BFS

Mã giả

```
bool vis[N];
ll par[N]={};
vector<ll> adj[N];
vector<ll> path(ll s,ll t){
    queue<ll> q;
    q.push(s);
    vis[s] = 1;
    while(!q.empty()){
        ll u = q.front();
        q.pop();
        for(ll v:adj[u]){
            if(!vis[v]){
                vis[v] = 1;
                par[v] = u;
                q.push(v);
            }
        }
    }
    vector<ll> res;
    if(!vis[t]) return res;
    for(ll v = t; v != s; v = par[v]) res.eb(v);
    res.eb(s);
    revers (res.begin(), res.end());
    eeturn res;
}
```

PATH by BFS

→ Tương tự như DFS, đường đi được trace lại phải lấy đảo ngược vì chúng ta đang đi ngược từ đỉnh đến đến đỉnh ban đầu

PATH by BFS

- → Tương tự như DFS, đường đi được trace lại phải lấy đảo ngược vì chúng ta đang đi ngược từ đỉnh đến đến đỉnh ban đầu
- → Nếu muốn thì bạn có thể sort trước mảng danh sách kề, điều đó sẽ làm cho đường đi bạn lấy được là đường đi ngắn nhất có thể, còn nếu không, nó chỉ cho ra 1 đường đi bất kì phụ thuộc vào mảng danh sách kề của bạn hiện tại

PATH by BFS

- → Tương tự như DFS, đường đi được trace lại phải lấy đảo ngược vì chúng ta đang đi ngược từ đỉnh đến đến đỉnh ban đầu
- → Nếu muốn thì bạn có thể sort trước mảng danh sách kề, điều đó sẽ làm cho đường đi bạn lấy được là đường đi ngắn nhất có thể, còn nếu không, nó chỉ cho ra 1 đường đi bất kì phụ thuộc vào mảng danh sách kề của bạn hiện tại
- → Điểm khác biệt lớn nhất giữa DFS và BFS trong việc tìm đường đi đó là khi mảng danh sách kề được sort, nếu như DFS sẽ cho ra đường đi có thứ tự từ điển nhỏ nhất thì BFS sẽ luôn cho ra đường đi ngắn nhất có thể từ u đến v

Floyd-Warshall

Khái quát

→ Floyd-Warshall là thuật toán tính **tất cả cặp đường đi ngắn nhất** trong đồ thị có trọng số (có thể âm, nhưng không có chu trình âm)

Floyd-Warshall

Khái quát

- → Floyd-Warshall là thuật toán tính **tất cả cặp đường đi ngắn nhất** trong đồ thị có trọng số (có thể âm, nhưng không có chu trình âm)
- → Sử dụng cơ sở **dynamic programming(DP)**:
  dist[i][j] = chi phí đường đi ngắn nhất từ i đến j

Floyd-Warshall

Khái quát

- → Floyd-Warshall là thuật toán tính **tất cả cặp đường đi ngắn nhất** trong đồ thị có trọng số (có thể âm, nhưng không có chu trình âm)
- → Sử dụng cơ sở **dynamic programming(DP)**:
  dist[i][j] = chi phí đường đi ngắn nhất từ i đến j
- → Khác với DFS/BFS, Floyd-Warshall không dựa trên duyệt sâu hay rộng, mà cập nhật từ các đỉnh trung gian

Floyd-Warshall

Mã giả

```
ll n,dist[N][N];
void floydWarshal () {
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] != oo && dist[k][j] != oo) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }
}
```

Floyd-Warshall

→ như vậy, ta có thể thấy rằng, floyd là một thuật toán khá đơn giản để tìm đường đi ngắn nhất giữa 2 đỉnh bất kì trong đồ thị với độ phức tạp khi lấy kết quả chỉ đơn giản là O(1) do dựa trên cách hoạt động của DP.

Floyd-Warshall

- → như vậy, ta có thể thấy rằng, floyd là một thuật toán khá đơn giản để tìm đường đi ngắn nhất giữa 2 đỉnh bất kì trong đồ thị với độ phức tạp khi lấy kết quả chỉ đơn giản là O(1) do dựa trên cách hoạt động của DP.
- → Tuy nhiên, điểm yếu lớn nhất của thuật toán này đó là độ phức tạp $O(n^3)$ khi phải cập nhật tất cả cặp đường đi ngắn nhất trong đồ thị, điều này làm cho nó không phù hợp với các đồ thị lớn

Floyd-Warshall

- → như vậy, ta có thể thấy rằng, floyd là một thuật toán khá đơn giản để tìm đường đi ngắn nhất giữa 2 đỉnh bất kì trong đồ thị với độ phức tạp khi lấy kết quả chỉ đơn giản là O(1) do dựa trên cách hoạt động của DP.
- → Tuy nhiên, điểm yếu lớn nhất của thuật toán này đó là độ phức tạp $O(n^3)$ khi phải cập nhật tất cả cặp đường đi ngắn nhất trong đồ thị, điều này làm cho nó không phù hợp với các đồ thị lớn
- → Để khắc phục điểm yếu này, chúng ta có thể sử dụng các thuật toán khác như Dijkstra hoặc Bellman-Ford để tìm đường đi ngắn nhất từ một đỉnh nguồn đến tất cả các đỉnh khác, sau đó lặp lại quá trình này cho mỗi đỉnh trong đồ thị để có được tất cả cặp đường đi ngắn nhất với độ phức tạp $O(n \cdot (E + V \log V))$ khi sử dụng Dijkstra với hàng đợi ưu tiên.

Dijkstra

Khái quát

→ Dijkstra là thuật toán tìm **đường đi ngắn nhất từ 1 nguồn** u đến tất cả các đỉnh còn lại trong đồ thị có trọng số không âm.

Dijkstra

Khái quát

→ Dijkstra là thuật toán tìm **đường đi ngắn nhất từ 1 nguồn** u đến tất cả các đỉnh còn lại trong đồ thị có trọng số không âm.

→ Sử dụng cấu trúc **priority queue / heap** để luôn chọn đỉnh có khoảng cách ngắn nhất chưa được duyệt.

Dijkstra

Khái quát

- → Dijkstra là thuật toán tìm **đường đi ngắn nhất từ 1 nguồn** u đến tất cả các đỉnh còn lại trong đồ thị có trọng số không âm.
- → Sử dụng cấu trúc **priority queue / heap** để luôn chọn đỉnh có khoảng cách ngắn nhất chưa được duyệt.
- → Tương tự DFS/BFS, chúng ta cần mảng **dist[]** lưu khoảng cách ngắn nhất và mảng **par[]** để trace đường đi.

Dijkstra

Khái quát

- → Dijkstra là thuật toán tìm **đường đi ngắn nhất từ 1 nguồn** u đến tất cả các đỉnh còn lại trong đồ thị có trọng số không âm.
- → Sử dụng cấu trúc **priority queue / heap** để luôn chọn đỉnh có khoảng cách ngắn nhất chưa được duyệt.
- → Tương tự DFS/BFS, chúng ta cần mảng **dist[]** lưu khoảng cách ngắn nhất và mảng **par[]** để trace đường đi.
- → Khi cập nhật đường đi: nếu đi qua nút u đến v mà dist[u] + w(u,v) < dist[v] thì cập nhật dist[v] và par[v] = u.

Dijkstra

Mã giả

```
ll n,dist[N], par[N];
vector<pair<int,int>> adj[N];
void dijkstra(int s) {
    for(int i=1;i<=n;i++) {
        dist[i] = oo;
        par[i] = -1;
    }
    dist[s] = 0;
    priority_queu <pair<int,int>, vector<pair<int,int>>, greate <pair<int,int>>> pq;
    pq.push({0, s});                                                r
    while(!pq.empty()) {
        int d = pq.top().first;
        int u = pq.top().second;
        pq.pop();
        if(d > dist[u]) continue; // đã có khoảng cách tốt hơn
        for(auto e : adj[u]) {
            int v = e.first;
            int w = e.second;
            if(dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                par[v] = u;
                pq.push({dist[v], v});
            }
        }
    }
}
```

Dijkstra

Tổng quát

→ so với phương phát floyd, Dijkstra chỉ tập trung vào việc tìm đường đi ngắn nhất từ 1 nguồn đến tất cả các đỉnh còn lại, trong khi Floyd-Warshall tính tất cả cặp đường đi ngắn nhất.

Dijkstra

Tổng quát

- → so với phương phát floyd, Dijkstra chỉ tập trung vào việc tìm đường đi ngắn nhất từ 1 nguồn đến tất cả các đỉnh còn lại, trong khi Floyd-Warshall tính tất cả cặp đường đi ngắn nhất.
- → đánh đổi cho sự bất tiện khi chỉ có thể tìm đường đi ngắn nhất từ 1 nguồn, Dijkstra có độ phức tạp thấp hơn nhiều so với Floyd-Warshall, đặc biệt là trên các đồ thị thưa (sparse graphs).

Dijkstra

Tổng quát

- → so với phương phát floyd, Dijkstra chỉ tập trung vào việc tìm đường đi ngắn nhất từ 1 nguồn đến tất cả các đỉnh còn lại, trong khi Floyd-Warshall tính tất cả cặp đường đi ngắn nhất.
- → đánh đổi cho sự bất tiện khi chỉ có thể tìm đường đi ngắn nhất từ 1 nguồn, Dijkstra có độ phức tạp thấp hơn nhiều so với Floyd-Warshall, đặc biệt là trên các đồ thị thưa (sparse graphs).
- → với độ phức tạp $O((V + E) \log V)$ khi sử dụng heap, Dijkstra thường được ưa chuộng trong các ứng dụng thực tế như định tuyến mạng, bản đồ, và các hệ thống GPS cũng như đây là một trong những thuật toán tuyệt vời nhất để tìm đường đi ngắn nhất trong đồ thị có trọng số không âm.

Dijkstra - Lưu ý

Điểm khác biệt với DFS/BFS

- → DFS: chỉ tìm 1 đường đi, không đảm bảo ngắn nhất về số bước hay trọng số.
- → BFS: tìm đường đi ngắn nhất về số bước, nhưng chỉ áp dụng khi trọng số bằng 1.
- → Dijkstra: tìm đường đi ngắn nhất theo trọng số >=0.
- → Floyd-Warshall: tìm tất cả cặp đường đi ngắn nhất, dùng DP, độ phức tạp $O(n^3)$.

Cảm ơn thầy cô và các bạn đã theo dõi!

Xin chân thành cảm ơn